BOTO3 SERIES | LAB 4

DynamoDB Student Manager

A database with no servers, no SQL and no joins — and conditional writes that enforce your rules.

DURATION	SERVICES	DELIVERABLE	PATTERN
2 h	**DynamoDB**	**dynamodb_manager.py**	**Conditions & error codes**

1 Goal

Build a menu-driven student records system backed by DynamoDB. It creates the table, adds students without allowing duplicates, reads them back by key, updates single fields, deletes records, lists everything with a paginated Scan, and searches with a filter. At the end, it deletes the table.

ARCHITECTURE

```
Python Application
        |
      Boto3
        |
    DynamoDB
```

PREVIEW OF THE OUTPUT

```
========================================
STUDENT MANAGER
========================================
1. Add Student           6. Search Students
2. Get Student           7. Query demo
3. Update Student        8. Create Table
4. Delete Student        9. DELETE Table
5. List Students         0. Exit
========================================
```

2 Learning objectives

- Explain the difference between a partition key and a composite primary key.
- Say why `boto3.resource` is preferred over `boto3.client` for DynamoDB.
- Create a table with on-demand billing and wait until it is `ACTIVE`.
- Use a `ConditionExpression` so the database, not your Python, enforces uniqueness.
- Alias reserved words with `ExpressionAttributeNames`.
- Explain the difference between Query and Scan, and what a `FilterExpression` does and does not save.

3 Before you start

- [ ] Labs 1 to 3 run successfully.
- [ ] You have re-copied your credentials for this session.
- [ ] You have chosen a table name with your own prefix (e.g. `youssef_Students`).
- [ ] You know that DynamoDB numbers are `Decimal`, never `float`.

Learner Lab reminder

- Always use `BillingMode="PAY_PER_REQUEST"`. An idle on-demand table costs essentially nothing, and there is no capacity to plan.
- Give your table a name with your own prefix so thirty students do not share one table.
- Delete the table at the end of the lab.

4 Step by step

Use the resource interface

Use `boto3.resource` for everything in this lab. The client interface makes you wrap every value in its type descriptor by hand.

Step 1: Create the table, and wait for it

Declare only the key attributes in `AttributeDefinitions`. Every other attribute is schemaless: that is the NoSQL part, and students routinely try to declare them all.

Creation takes 10 to 20 seconds. Without the waiter, your very next `put_item` raises `ResourceNotFoundException`, which looks exactly like a name typo and is not.

```
import boto3
from botocore.exceptions import ClientError

TABLE_NAME = "youssef_Students"
dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table(TABLE_NAME)

def create_table():
    try:
        new_table = dynamodb.create_table(
            TableName=TABLE_NAME,
            KeySchema=[
                {"AttributeName": "student_id", "KeyType": "HASH"},
            ],
            AttributeDefinitions=[
                {"AttributeName": "student_id", "AttributeType": "S"},
            ],
            BillingMode="PAY_PER_REQUEST",
        )
        print("Creating table, please wait ...")
        new_table.wait_until_exists()
        print(f"Table '{TABLE_NAME}' is ACTIVE.")
    except ClientError as e:
        if e.response["Error"]["Code"] == "ResourceInUseException":
            print("Table already exists.")
        else:
            raise
```

Checkpoint

The Console shows the table with status Active and on-demand capacity. Running option 8 a second time says it already exists rather than crashing.

Step 2: Add a student, and refuse duplicates

`put_item` overwrites by default. If `S001` already exists, a plain `put_item` replaces the whole item and you never find out.

The naive fix is to `get_item` first and check. That has a race condition: another process can write between your check and your write. The correct fix is one atomic call with the condition inside it.

Numbers must be `Decimal`. Passing a `float` raises `TypeError` before the request is even sent.

```
from decimal import Decimal

def add_student():
    item = {
        "student_id": input("Student ID: ").strip(),
        "name": input("Full name: ").strip(),
        "email": input("Email: ").strip(),
        "program": input("Program: ").strip(),
    }
    year = input("Year (number): ").strip()
    if year.isdigit():
        item["year"] = Decimal(year)

    try:
        table.put_item(
            Item=item,
            ConditionExpression="attribute_not_exists(student_id)",
        )
        print(f"Added student {item['student_id']}")
    except ClientError as e:
        if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
            print("That student already exists. Use Update instead.")
        else:
            raise
```

> **Checkpoint**
>
> Adding `S001` twice prints a clear message the second time, and the first record is unchanged. Verify that in the Console.

Step 3: Read one student

`get_item` is the cheapest read DynamoDB offers: it goes straight to one partition using the key. Whenever you know the key, use it.

A miss is not an error. The response simply has no `Item` key at all.

```
def get_student():
    student_id = input("Student ID: ").strip()
    response = table.get_item(Key={"student_id": student_id})
    if "Item" not in response:
        print(f"No student with ID {student_id}")
        return
    for key, value in response["Item"].items():
        print(f"  {key:<14}: {value}")
```

> **Checkpoint**
>
> Requesting a student who does not exist prints a message, not a `KeyError`.

Step 4: Update a single field

`UpdateExpression` modifies attributes in place without rewriting the item. Use it whenever you want to change one field: `put_item` would silently erase every attribute you left out.

`ExpressionAttributeNames` exists because DynamoDB reserves over 570 words, including `name`, `year`, `status`, `size` and `count`. Aliasing them with `#` is mandatory, not stylistic.

Add `attribute_exists` as a condition so a typo in the ID does not silently create a new item.

```
def update_student():
    student_id = input("Student ID: ").strip()
    field = input("Field (name/email/program/year): ").strip()
    value = input("New value: ").strip()
    if field in ("year", "grade"):
        value = Decimal(value)

    try:
        response = table.update_item(
            Key={"student_id": student_id},
            UpdateExpression="SET #field = :value",
            ExpressionAttributeNames={"#field": field},
            ExpressionAttributeValues={":value": value},
            ConditionExpression="attribute_exists(student_id)",
            ReturnValues="ALL_NEW",
        )
        print("Updated:", response["Attributes"])
    except ClientError as e:
        if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
            print(f"No student with ID {student_id}")
        else:
            raise
```

> **Checkpoint**
>
> Updating the `name` field works. Remove the `ExpressionAttributeNames` alias and confirm it fails, then put it back. That failure is worth seeing once.

Step 5: Delete a student

Use `ReturnValues="ALL_OLD"` so you can tell the user which record was removed, and a condition so deleting a non-existent ID reports instead of silently succeeding.

```
response = table.delete_item(
    Key={"student_id": student_id},
    ConditionExpression="attribute_exists(student_id)",
    ReturnValues="ALL_OLD",
)
print(f"Deleted {response.get('Attributes', {}).get('name', student_id)}")
```

> **Checkpoint**
>
> Deleting a student prints their name. Deleting a made-up ID says so.

Step 6: List everything with a paginated Scan

`Scan` reads every item in the table and is paginated by `LastEvaluatedKey`. Handle that loop or you silently get only the first 1 MB of your table: the classic silent data-loss bug.

This loop is the manual version of what a paginator does. Writing it once makes the paginator make sense.

```
def list_students():
    items = []
    scan_kwargs = {}
    while True:
        response = table.scan(**scan_kwargs)
        items.extend(response.get("Items", []))
        if "LastEvaluatedKey" not in response:
            break
        scan_kwargs["ExclusiveStartKey"] = response["LastEvaluatedKey"]

    for item in sorted(items, key=lambda x: x["student_id"]):
        print(f"{item['student_id']:<12} {item.get('name', '-'):<24} "
              f"{item.get('program', '-')}")
    print(f"\n{len(items)} student(s)")
```

> **Checkpoint**
>
> All your students are listed, sorted by ID.

Step 7: Search, and understand what it costs

`FilterExpression` is applied after the scan has read the items. It reduces what you receive over the network, but it does not reduce what you pay for. That distinction is the classic DynamoDB exam question.

`Key()` conditions work only on key attributes; `Attr()` works on anything. Using `Attr()` in a `KeyConditionExpression` is a validation error, and everyone makes it once.

```
from boto3.dynamodb.conditions import Attr, Key

# Scan + filter: reads the whole table, returns the matches
response = table.scan(
    FilterExpression=Attr("program").contains(term)
)

# Query: goes straight to one partition. Fast and cheap.
response = table.query(
    KeyConditionExpression=Key("student_id").eq(student_id)
)
```

> **Checkpoint**
>
> You can state, in one sentence, why Query is cheaper than Scan + Filter even when both return one item.

5 Expected output

Your formatting may differ, but the information must be the same.

EXAMPLE SESSION

```
Choose an option: 1
Student ID: S001
Full name: Ana Silva
Email: ana@example.com
Program: Computer Science
Year (number): 2
Added student S001

Choose an option: 1
Student ID: S001
...
That student already exists. Use Update instead.

Choose an option: 5
ID          NAME                    PROGRAM            YEAR
------------------------------------------------------------
S001        Ana Silva               Computer Science   2
S002        Karim Ben Ali           Data Science       1

2 student(s)
```

6 Troubleshooting

You see	It means	Do this
`ResourceNotFoundException`	Table not ready, or wrong name or region.	Wait with `wait_until_exists()`; check the name.
`ResourceInUseException`	The table already exists.	Use it, or delete it first.
`ConditionalCheckFailedException`	Your condition held, as designed.	Catch it and tell the user. It is not a crash.
`ValidationException`	Reserved word, or `Key()`/`Attr()` misuse.	Alias with `#`; use `Key()` only on key attributes.
`TypeError: Float types ...`	You passed a Python float.	Wrap it: `Decimal("16.5")`, from a string.
`Object of type Decimal ...`	`json.dumps` cannot serialise `Decimal`.	Pass `default=str`, or convert before dumping.
`KeyError: 'Item'`	`get_item` found nothing.	Check for `"Item"` in the response first.

7 Cleanup

- [ ] Your DynamoDB table is deleted (option 9).
- [ ] The DynamoDB Console shows no tables belonging to you.
- [ ] Your script is committed to your Git repository.

8 Submission checklist

- [ ] `dynamodb_manager.py` runs end to end with no traceback.
- [ ] The table is created on-demand and the script waits until it is `ACTIVE`.
- [ ] Adding the same student twice is refused by a `ConditionExpression`, and the error is caught.
- [ ] Update, delete, list (with `LastEvaluatedKey` pagination), search and query all work.

- [ ] Numbers are stored as `Decimal`, and reserved words are aliased with `#`.
- [ ] The file is committed with a meaningful message.